

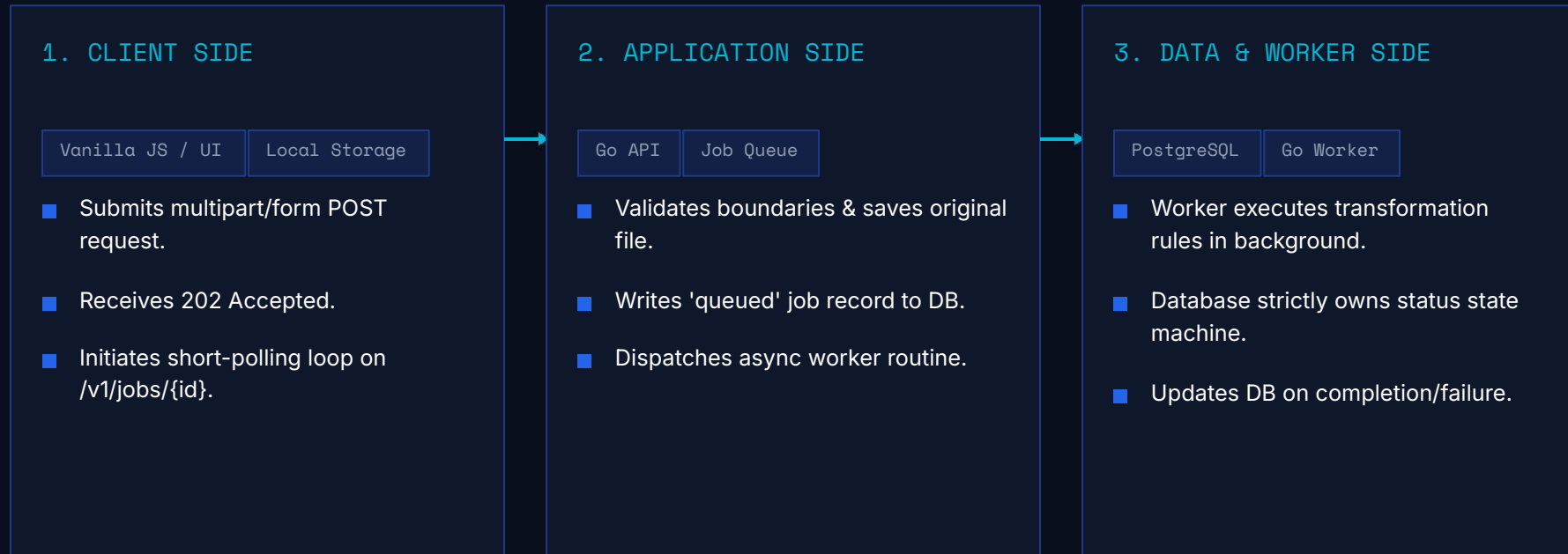
ImageLab Architecture & Week 4 Demonstration

Asynchronous Processing & Short Polling (CMPS 4191)

>

Asael Tobar and Tysha Daniels

Architectural Mental Model



PRINCIPLE: Request accepts work; Worker executes; Postgres owns state.

HTTP 202 Contract

REQUEST BOUNDARY

ENDPOINT

```
POST /v1/images
```

CONTENT-TYPE

```
multipart/form-data
```

- Validate format (JPEG / PNG only).
- Enforce max file size (10MB).
- Store original binary file securely.
- Create tracking record in database.

STRICT RULE: The HTTP handler thread must NEVER block to generate image variants.

RESPONSE PAYLOAD

STATUS CODE

```
202 ACCEPTED
```

LOCATION HEADER

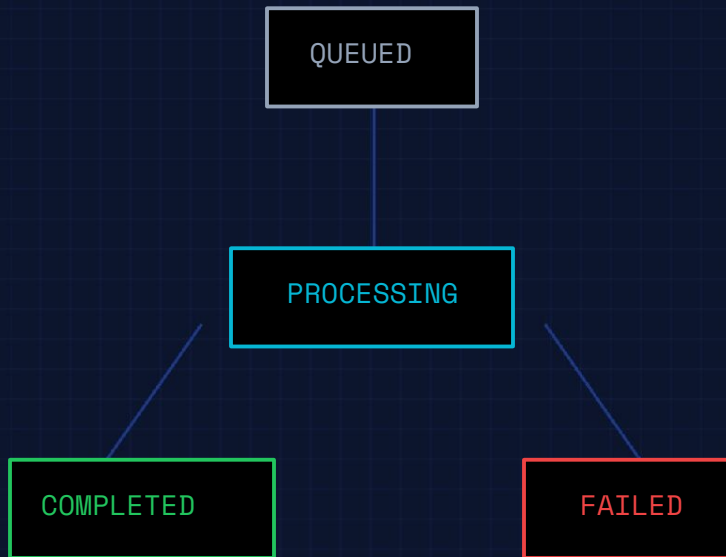
```
/v1/jobs/144a8c9b
```

BODY (JSON)

```
{
  "image_id": "img_939a",
  "job_id": "144a8c9b",
  "status": "queued",
  "status_url": "/v1/jobs/144a8c9b"
}
```

Worker & Transformations

Go Single Worker Thread



Required Variants

Thumbnail

150×150 pixel exact crop

Preview

Max 800×600 resolution

Display

Max 1200×900 resolution

COMPLETION RULE: The job transitions to **COMPLETED** state strictly *after* all 3 variants are securely stored on disk.

Short Polling Mechanics

T+0.0s

POST /v1/images -> 202 Location: /v1/jobs/144a

Upload accepted. JS initiates ~1 second timeout loop.

T+1.0s

GET /v1/jobs/144a -> status: "processing"

Server returns immediate state. Client schedules next GET.

T+2.0s

GET /v1/jobs/144a -> status: "completed"

Terminal state reached. Polling loop stops immediately. UI renders result.

Abort Mechanism: Polling utilizes `AbortController` to cleanly terminate in-flight requests on page unload or new uploads.

UI Protection & Error Distinction

DOUBLE-SUBMIT GUARD

The UI relies on an `isSubmitting` flag and disabled button state to prevent overlapping POST requests while a file is uploading or waiting for worker processing.

Retrieval Error

NETWORK / TIMEOUT

Job state observation failed (e.g., polling endpoint returns 503 or times out).

ACTION: Stop polling. Preserve job ID.

Show: "Unable to check status"

Provide: "Try Again" manual check button.

Processing Failure

status = failed

Observation succeeded. The worker explicitly marked the job as unprocessable.

ACTION: Stop polling. Discard job state.

Show: "Image processing failed."

Require fresh upload.

Week 4 Integration Metrics

Experiment Scope: Comparison of observed latencies between a **Single Image** submission and a **5-Job Concurrent Burst** hitting a single background worker.

Metric	Description	Single Image	5-Job Burst
Ack Latency	Time from POST to 202 Location	Immediate	Immediate
Queue Wait	Time spent in 'queued' state	Minimal	Observable queuing delay
Process Time	Time from 'queued' to 'completed'	Baseline	Baseline per job
Job Duration	Total time (Queue Wait + Process Time)	Baseline	Linear growth (1 worker)
Polling Count	Total GET requests per job	Low (~1-2)	High (waits in queue)
Detection Delay	Time between DB update and UI render	Max 1000ms	Max 1000ms

System Analysis & Trade-offs

202 ACCEPTED + SINGLE WORKER

[+ ADVANTAGE

Significantly reduces HTTP acknowledgment latency. The client thread is never blocked waiting for file transformations.

[+ ADVANTAGE

Protects the application host from CPU/memory spikes during burst uploads.

[- LIMITATION

Total transformation work is not reduced. The single worker creates strict serial execution, causing observable queue waits during bursts.

SHORT POLLING CLIENT

[+ ADVANTAGE

Mechanically simple to implement. Requires zero persistent connections or complex server-side push infrastructure (WebSockets).

[- LIMITATION

Adds substantial HTTP traffic overhead, especially if the worker queue is backed up.

[- LIMITATION

Inherent detection delay. The UI state is permanently up to ~1.0 second behind the actual database state.


```
worker@imagedlab:~/submit$ ./verify_dod.sh
```

```
[ OK ] Complete end-to-end integration verified.
```

```
[ OK ] Error handling & status recovery functional.
```

```
[ OK ] Single-worker queuing confirmed via stress test.
```

```
[ OK ] Code, migrations, and clean commits prepared.
```

```
[ OK ] README updated with Week 4 instructions.
```

```
THANKS FOR LISTENING!
```